CS: 6359
Assignment 1: Finding 20 Code Smells

Submitted By: Jannatul Ferdoush Jannati
NetID: dal934908

Customer.java:

1. Line 6: `private Vector _rentals = new Vector();`
   Deprecated API Usage: Using *Vector* is a code smell here since it is an unsupported collection type or a legacy class.
2. Line 9, 13: `_name = name;`
   Non-Standard Naming Convention: Using underscores before variable names instead of camelCase is a code smell.
3. Line 20: `public String statement() {`
   Naming Smell: The name of the function is not compatible as the function returns the rental amount.
4. Line 20-71: `public String statement() {…}`
   Long Method: This function does more than one task like calculating frequentRenterPoints, rental amount, etc.
5. Line 30: `Rental each = (Rental) rentals.nextElement();`
   Variable Naming Issue: variable name 'each' is confusing when used like in line 33, 36, etc.
6. Line 33, 55-56: `switch (each.getMovie().getPriceCode()) {`
   Switch Statements: Choosing behavior based on the movie's price code with a switch is a smell, and it also has no default case.
7. Line 35, 37 (also 41, 44, 46): `thisAmount += 2;`
   Magic Numbers: The numbers 2, 3, and 1.5 are used directly with no name to explain what they mean.
8. Line 43-48: `thisAmount += 1.5;`
   Duplicated Code: The CHILDRENS case repeats the same structure as the REGULAR case with only the numbers changed.
9. Line 33, 55, 61: `each.getMovie().getPriceCode()`
   Message Chains: Calling each.getMovie().getPriceCode() chains through several objects, so Customer depends on their structure.
10. Line 23: `int frequentRenterPoints = 0;`
    Recomputed State: frequentRenterPoints is recalculated from zero on every call instead of being stored on the Customer.
11. Line 32, 51, 54, 60, 66: `// determine amounts for each line`
    Comments: Each block of the method needs a comment to explain it, which shows the method is doing too many things.
12. Line 20-71: `public String statement() {…}`
    Divergent Change: statement() has to be changed for two unrelated reasons, the pricing rules and the output format.
13. Line 33: `switch (each.getMovie().getPriceCode()) {`
    Shotgun Surgery: Adding a new movie category needs changes in several places, in both Movie and Customer.

14. Line 34, 40, 43, 55: Movie.REGULAR, Movie.NEW_RELEASE, Movie.CHILDRENS
    Inappropriate Intimacy: Customer depends directly on Movie's internal price-code constants.
15. Line 24, 27, 30: Enumeration rentals = _rentals.elements();
    Deprecated API Usage: Enumeration, hasMoreElements(), and nextElement() are legacy iteration APIs.

Movie.java:

16. Line 1: public class Movie {
    Data Class: Movie only holds data with getters and a setter and has no behavior of its own.
17. Line 3-5: public static final int CHILDRENS = 2;
    Primitive Obsession: The movie categories are stored as plain int constants, so a movie's type is just a number with no type safety.
18. Line 19-21: public void setPriceCode(int arg) {…}
    Speculative Generality: setPriceCode() is a public setter that is never used anywhere in the code.

Rental.java:

19. Line 1: public class Rental {
    Data Class: Like Movie, Rental only stores data behind getters and has no behavior of its own.
20. Line 6, 7: _movie = movie;
    _daysRented = daysRented;
    Non-Standard Naming Convention: Using underscores before variable names instead of camelCase is a code smell.